

外部中断实验

外部中断实验

一、学习目标

外部中断

外部中断优点及作用

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. 配置引脚模式

3. 写入程序

4. 编译

四、程序分析

五、实验现象

一、学习目标

1.学习MSPM0G3507主板外部中断的知识。

2.使用板载按键控制板载led灯的开关。

外部中断

中断就是cpu在执行程序时，有其他事件需要占用cpu，cpu接收到这个中断请求后暂停原程序的执行，去执行中断函数，中断函数执行完成之后，继续回到原来的程序继续执行。

外部中断（External Interrupt）是指由外部硬件事件（通常是来自微控制器外部的信号）触发的中断。这种中断可以打断微控制器当前的程序执行流程，从而迅速响应外部的硬件事件。

外部中断的触发通常与特定的引脚（GPIO引脚）相关联，例如按键、传感器信号、定时器信号等。这些引脚通常被配置为检测特定的信号变化（如上升沿、下降沿或电平变化），从而触发中断。

外部中断优点及作用

优点：

- 快速响应：外部中断能够在触发事件发生时立即响应，而无需轮询信号。
- 节省资源：不需要在主循环中不断检测信号状态，节省了CPU资源。
- 提高实时性：中断机制能够确保对关键事件的及时处理，从而提高系统的实时性。

作用：

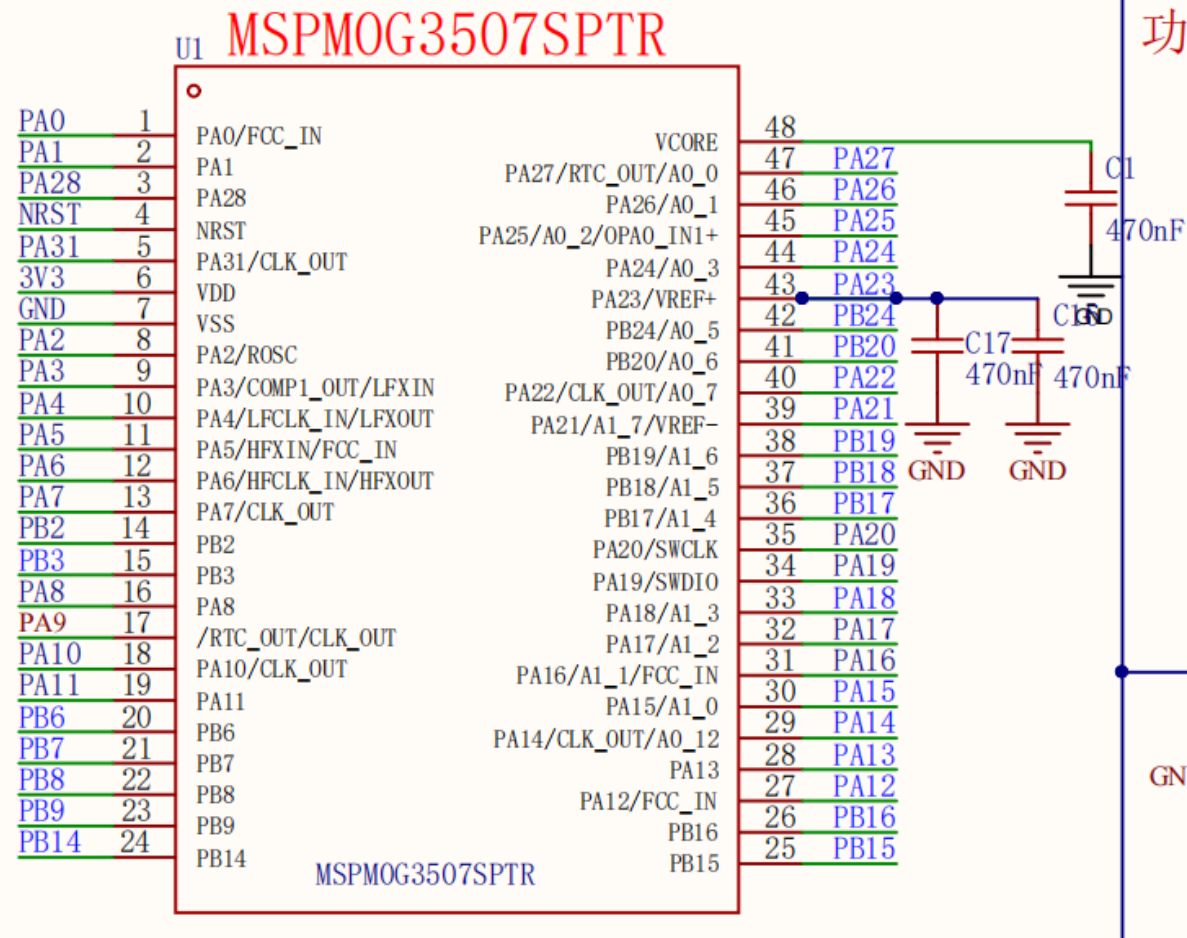
- 快速响应外部事件：可以在发生硬件事件时立即中断当前程序的执行并快速处理该事件，而不需要轮询检查信号状态。避免了程序对事件的轮询，大大节省了CPU资源，并且提高了系统对外部事件的响应速度，
- 提高系统实时性：在需要实时响应的场景中，外部中断可以确保关键事件被及时处理，无需等待系统主程序完成当前任务，从而提高系统的实时性能。
- 减轻主程序负担：外部中断能够将将与外部事件相关的任务从主程序中分离出来，避免了在主程序中轮询检查信号状态，提高了主程序的执行效率，使主程序可以专注于核心任务。
- 事件驱动系统的核心：可以实现高效的事件驱动机制，降低系统复杂度。

- 实现硬件交互和控制：外部中断使得微控制器能够实时处理和响应外部硬件的信号，确保系统正常运行。
- 数据采集和处理：在许多实时数据采集系统中，外部中断用于触发数据采集或处理。
- 低功耗场景的触发机制：通过外部中断，可以实现高效的低功耗设计，使系统在非工作时保持休眠状态，仅在必要时才运行。
- 保障系统安全：及时处理安全相关的事件，保障系统的可靠性和安全性。

二、硬件搭建

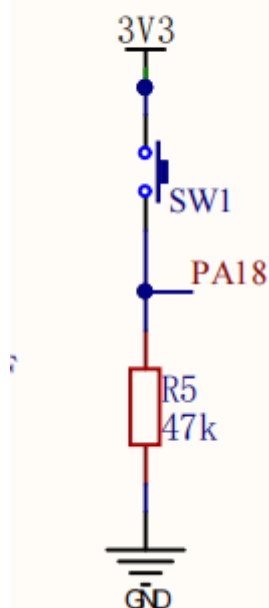
本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载功能按键。

MSPM0G3507主控图



按键部分原理图

功能按键

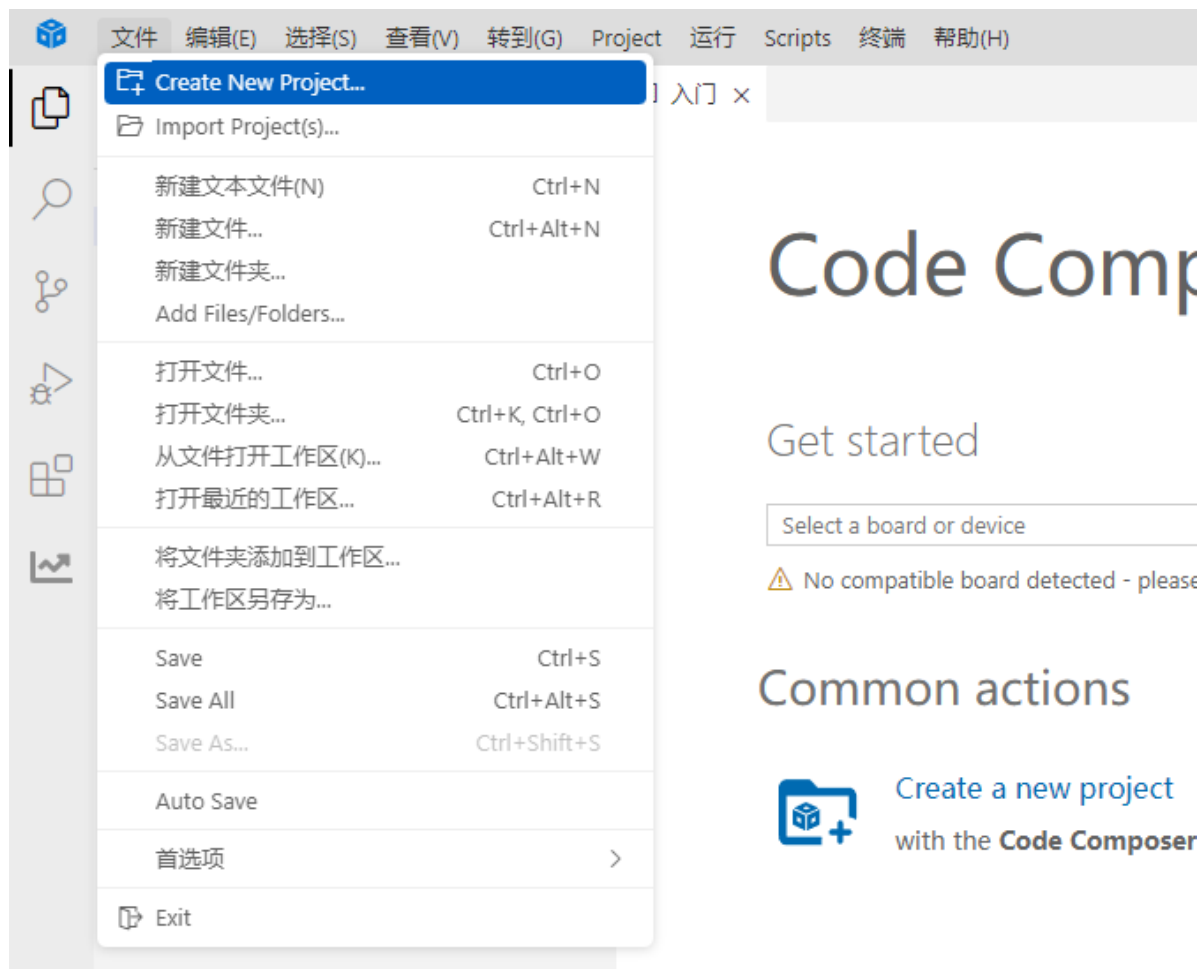


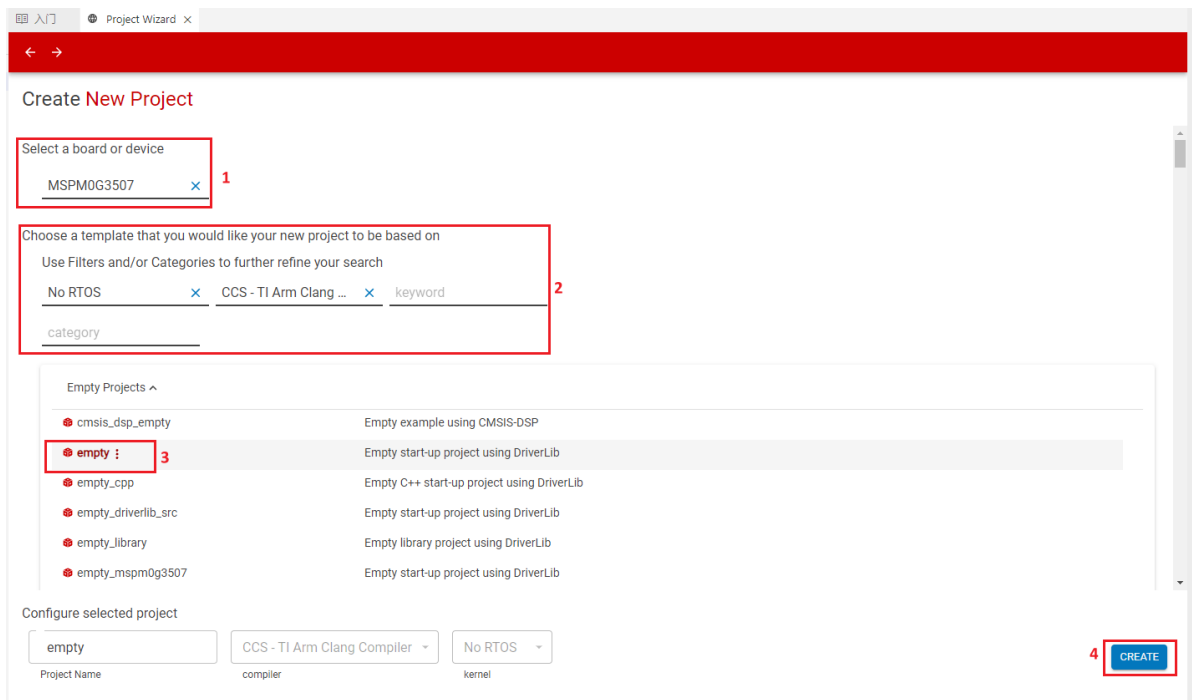
三、实验步骤

本节课中，将使用PA18按键触发PB2的LED1点亮和熄灭，软件上使用CPU中断方式。LED1的PB2引脚的配置请参考 点亮LED灯 课程。

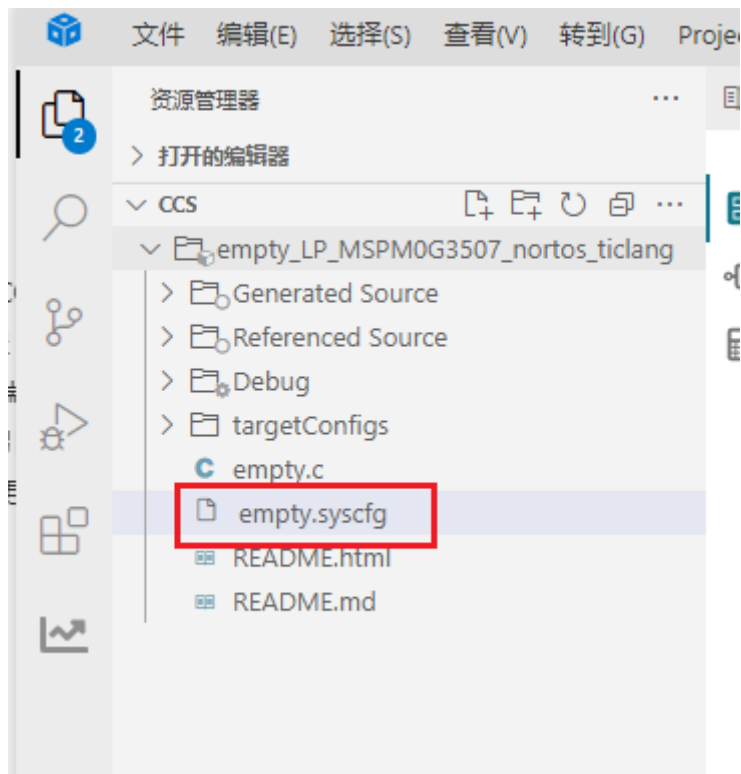
1. 打开SYSCONFIG配置工具

在CCS中新建一个空白工程 empty。

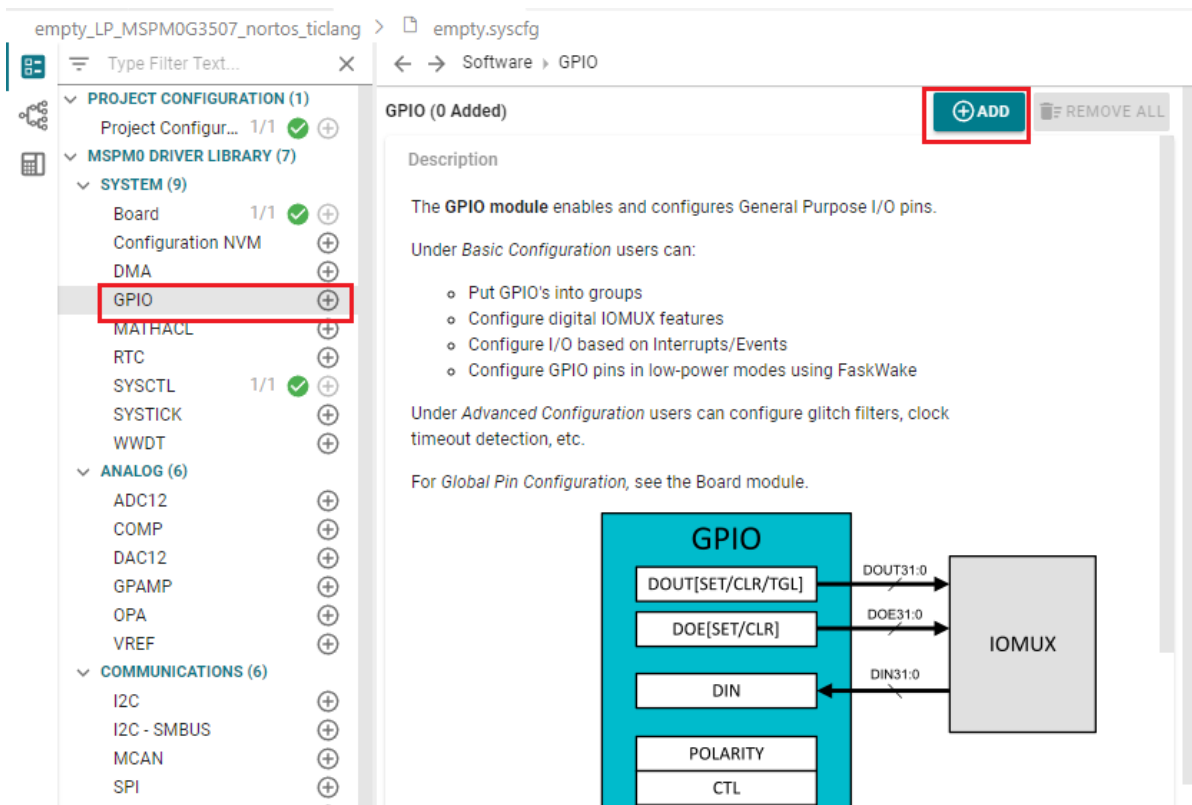




在CCS的左侧工作区中找到并打开empty.syscfg文件。



在SYSCONFIG中，左侧选择MCU外设，找到GPIO选项并点击进入。在GPIO中点击ADD，添加两组引脚外设，一组LED引脚，一组按键引脚。LED引脚参考前面点亮LED的课程，这里只讲按键引脚的配置。



2. 配置引脚模式

配置引脚参数，将GPIO实例命名为 **KEY**，由于接入的是 PA18，所以 Port 选择 **PORTA**。

引脚名称设置为 **PIN_18**，设置为输入模式 **Input**。Internal Resistor设置为下拉电阻 **Pull-Down Resistor**，Assigned Pin设置要控制的引脚为18。

KEY

Name	KEY
Port	PORTA
Port Segment	Any

Group Pins

1 added

+

 ADD

REMOVE ALL

KEY

PIN_18

Name	PIN_18
Direction	Input
IO Structure	Any

Digital IOMUX Features

Internal Resistor	Pull-Down Resistor
Invert	Disabled
Input Filter	Disabled
Enable Fast-Wake	☐
Override Pin Hysteresis Default	☐
Hysteresis Control	Disabled
Wakeup Logic	Disabled

Assigned Port	PORTA
Assigned Port Segment	Any
Assigned Pin	18

Interrupts/Events

LaunchPad-Specific Pin	No Shortcut Used
------------------------	------------------

PinMux Peripheral and Pin Configuration

PIN_18	Any(PA18/11)
--------	--------------

配置引脚外部中断。芯片上引出的所有GPIOA端口，都可以设置外部中断功能。

Enable Interrupts开启了中断，**Interrupt Priority**设置中断优先级为默认，当多个中断同时触发时，按芯片默认的优先级规则处理。

Trigger Polarity中断的触发方式为上升沿触发，按键按下时，GPIO 引脚的电平从低电平（0）变化为高电平（1），这一过程称为上升沿。这意味着外部中断将在引脚状态从低到高的变化时被触发。

Event Publishing channel事件发布通道不开启。事件发布通道是一种将中断事件发布到指定通道的机制，其他外设可以通过订阅通道上的事件来响应。

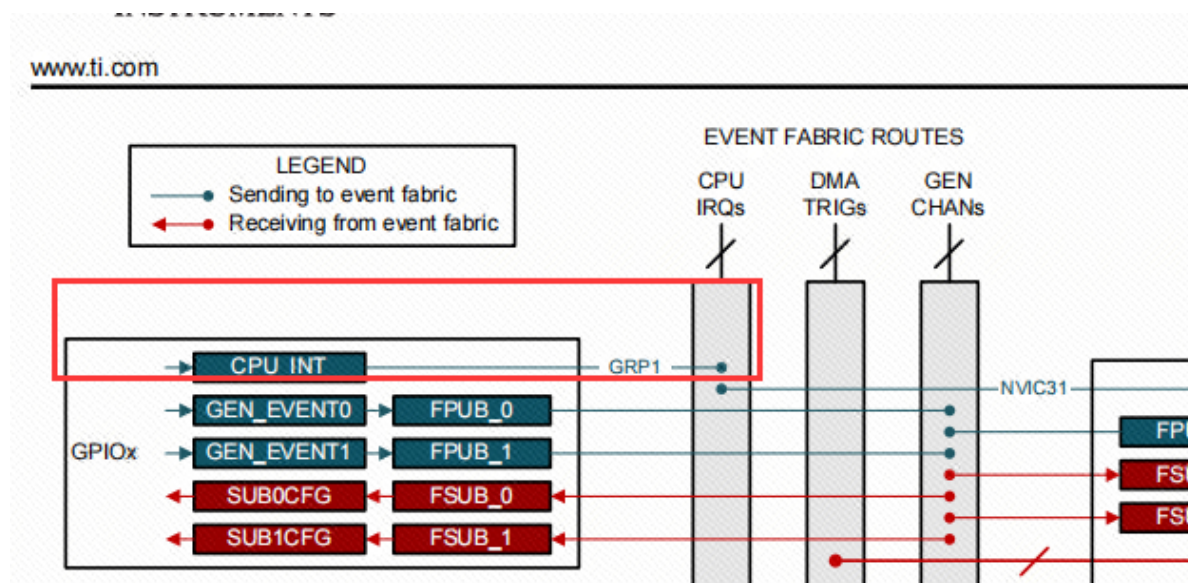
Interrupts/Events

Enable Interrupts	☒
Interrupt Priority	Default
Trigger Polarity	Trigger on Rising Edge
Event Publishing Channel	Disabled (0)

通过快捷键 Ctrl+S 将.syscfg文件中的配置保存。

3. 写入程序

根据MSPM0G系列的用户手册，可以知道GPIO的中断触发后，都是通过 **GRP1线** 将中断发布到总线，总线识别到之后就进入中断服务函数中执行内容。



配置好引脚的中断之后，我们还要手动编写外部中断的中断服务函数。因为我们开启了引脚下降沿中断，当我们的按键从按下到松开的时候产生的下降沿，就会触发一次中断，触发后就会执行中断服务函数。每个中断对应的中断服务函数名称通常是固定的，不能随意修改，否则将无法正确进入中断服务函数。中断服务函数的具体名称为 **GROUP1_IRQHandler**。

在empty.c文件中，写入以下代码

```
#include "ti_msp_dl_config.h"

int main(void)
{
    SYSCFG_DL_init();
    NVIC_EnableIRQ(KEY_INT_IRQN); //开启按键引脚的GPIOA端口中断 Enable GPIOA port
    interrupt for button pin

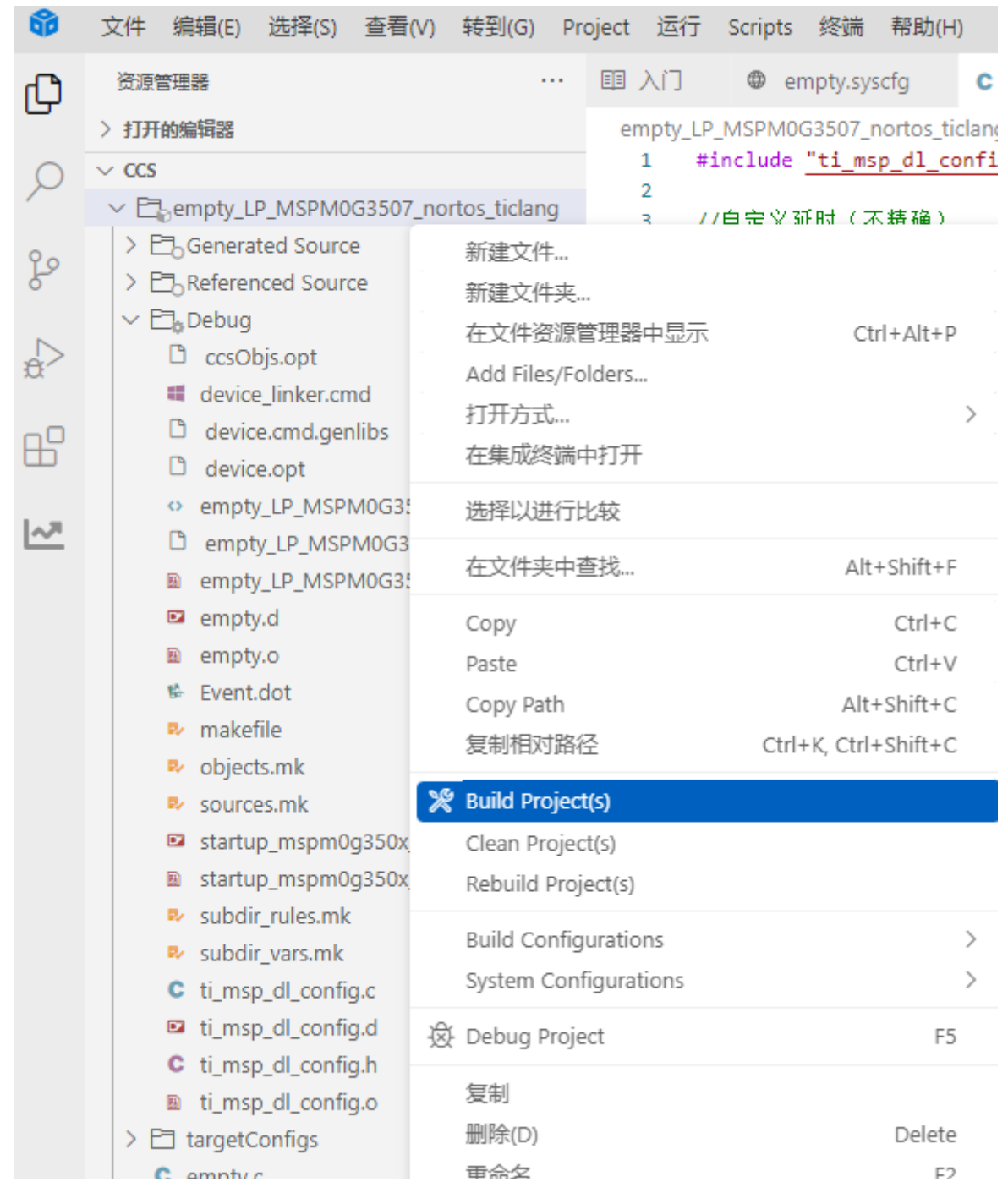
    while (1)
    {

    }
}

void GROUP1_IRQHandler(void) //Group1的中断服务函数 Interrupt service function of
Group1
{
    //读取Group1的中断寄存器并清除中断标志位
    // Read the interrupt register of Group1 and clear the interrupt flag
    switch( DL_Interrupt_getPendingGroup(DL_INTERRUPT_GROUP_1) )
    {
        //检查是否是KEY的GPIOA端口中断, 注意是INT_IIDX, 不是PIN_18_IIDX
        // Check if it is the GPIOA port interrupt of KEY, note that it is
        INT_IIDX, not PIN_18_IIDX
        case KEY_INT_IIDX:
            //如果按键按下变为高电平 If the button is pressed, it becomes high level
            if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
            {
```

```
//设置LED引脚状态翻转 Set LED pin state toggle
DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
}
break;
}
}
```

4. 编译



编译成功了，即可将程序下载到开发板。

四、程序分析

- empty.c

```

04_extInterrupt > C empty.c > GROUP1_IRQHandler
1  #include "ti_msp_dl_config.h"
2
3  int main(void)
4  {
5      SYSCFG_DL_init();
6      NVIC_EnableIRQ(KEY_INT_IRQN); //开启按键引脚的GPIOA端口中断 Enable GPIOA port interrupt for button pin
7
8      while (1)
9      {
10
11     }
12 }
13
14
15 void GROUP1_IRQHandler(void) //Group1的中断服务函数 Interrupt service function of Group1
16 {
17     //读取Group1的中断寄存器并清除中断标志位
18     // Read the interrupt register of Group1 and clear the interrupt flag
19     switch( DL_Interrupt_getPendingGroup(DL_INTERRUPT_GROUP_1) )
20     {
21         //检查是否是KEY的GPIOA端口中断, 注意是INT_IIDX, 不是PIN_18_IIDX
22         // Check if it is the GPIOA port interrupt of KEY, note that it is INT_IIDX, not PIN_18_IIDX
23         case KEY_INT_IIDX:
24             //如果按键按下变为高电平 If the button is pressed, it becomes high level
25             if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
26             {
27                 //设置LED引脚状态翻转 Set LED pin state toggle
28                 DL_GPIO_togglePins(LED1_PORT, LED1_PIN_2_PIN);
29             }
30             break;
31     }
32 }

```

uint32_t DL_Interrupt_getPendingGroup(DL_INTERRUPT_GROUP group) 用于获取指定中断组中最高优先级的待处理中断索引。

group: 一个枚举类型 `DL_INTERRUPT_GROUP`，用于指定检查的中断组。

这段代码实现了一个基本的中断服务功能，主要是通过按键触发中断来控制一个LED的状态。

`main()` 函数初始化之后开启按键的GPIOA端口中断，`GROUP1_IRQHandler()` 中断服务函数获取Group1中最高优先级的挂起标志，检查是否发生了按键中断，然后调用`DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN)` 读取按键引脚的电平变化，如果变为高电平，则LED灯引脚状态发生翻转。

五、实验现象

程序下载完成之后，可以通过按键控制LED灯的状态，按一下LED灯亮，再按一下LED灯灭。